

Guía de Práctica

Consumo de *Web Services* REST con Angular 22

Aplicaciones Web | Ingeniería de Software | UTEQ

Angular 22 • Node.js ≥ 22 • TypeScript 6 • julio de 2026

Coherencia de versiones. Esta guía está escrita íntegramente para **Angular 22**, la versión estable publicada el 3 de junio de 2026, que exige **Node.js 22 o superior** y **TypeScript 6**, e incorpora como estables las API de datos basadas en señales (`httpResource`) y el `HttpClient` sobre *Fetch* de forma predeterminada [1-3]. Todos los ejemplos usan componentes *standalone*, inyección con `inject()` y arquitectura *zoneless*, que son el modelo recomendado por el equipo de Angular en 2026.

1. Objetivos

El objetivo general de esta práctica es que el estudiante consuma un *Web Service* REST desde una aplicación Angular 22, distinguiendo cuándo emplear la API reactiva basada en señales y cuándo el cliente HTTP tradicional, y resolviendo correctamente las restricciones de origen cruzado (CORS).

De forma específica, al finalizar la práctica el estudiante será capaz de: explicar con sus palabras qué es una API REST y cómo se mapea a los métodos HTTP; configurar el cliente HTTP de Angular en una aplicación *standalone*; implementar un servicio que realice las cuatro operaciones básicas (leer, crear, actualizar y eliminar); mostrar datos en la interfaz con `httpResource` y señales; manejar errores de red y de servidor; y diagnosticar y resolver un bloqueo por CORS mediante un *proxy* de desarrollo.

2. Prerrequisitos y preparación del entorno

Antes de empezar verifique que su equipo cumple la línea base de versiones, pues Angular 22 no funciona con Node.js 20 ni con TypeScript anterior a la 6 [2].

```
1 # 1. Comprobar la versión de Node (debe ser 22 o superior).
2 node --version          # p. ej. v22.14.0 (Node 20 ya NO es compatible)
3
4 # 2. Instalar/actualizar la CLI de Angular de forma global.
5 npm install -g @angular/cli@22
6
7 # 3. Confirmar la versión de la CLI y del framework.
8 ng version              # @angular/core y @angular/cli deben coincidir en 22.x
9
10 # 4. Crear un proyecto nuevo (standalone y zoneless por defecto en v22).
11 ng new consumo-rest --style=css --ssr=false
12 cd consumo-rest
13
14 # 5. Levantar el servidor de desarrollo.
15 ng serve -o             # abre http://localhost:4200 en el navegador
```

Listing 1: Verificación e instalación del entorno.

La opción `-o` de `ng serve` (*open*) abre automáticamente el navegador. El servidor de desarrollo recompila y recarga la página ante cada cambio guardado (*live reload*); no es un servidor de producción.

3. Marco conceptual: cada término que usará

Esta sección define, uno por uno, los conceptos que aparecen en el código. Léala aunque ya conozca algunos términos, porque la práctica los usa con precisión.

3.1. Web Service, API y REST

Un *Web Service* es un componente de software accesible por la red que ofrece funcionalidad a otras aplicaciones mediante un contrato bien definido. Una *API* (*Application Programming Interface*, interfaz de programación de aplicaciones) es ese contrato: el conjunto de operaciones que el servicio expone y las reglas para invocarlas.

REST (Representational State Transfer)

REST es un *estilo arquitectónico* para sistemas distribuidos definido por Roy Fielding en su tesis doctoral de 2000 [4]. No es un protocolo ni una biblioteca, sino un conjunto de restricciones: arquitectura cliente-servidor, comunicación *sin estado* (cada petición contiene toda la información necesaria), interfaz uniforme y manipulación de *recursos* a través de sus representaciones. Una API que respeta estas restricciones se denomina *RESTful*.

En REST, cada entidad relevante es un *recurso* identificado por una URL. Al camino que apunta a un recurso o a una colección se le llama *endpoint* (punto de acceso). Por ejemplo, `/api/productos` identifica la colección de productos y `/api/productos/7` identifica el producto cuyo identificador es 7.

3.2. HTTP: métodos y códigos de estado

REST se apoya en HTTP, cuyo significado y semántica están normalizados en el RFC 9110 [5]. La operación que se desea realizar sobre un recurso se expresa con el *método* (o *verbo*) HTTP.

Operación	Método HTTP	Endpoint típico	Significado
Leer (todos)	GET	<code>/api/productos</code>	Obtener la colección
Leer (uno)	GET	<code>/api/productos/7</code>	Obtener un recurso
Crear	POST	<code>/api/productos</code>	Añadir un recurso nuevo
Actualizar	PUT	<code>/api/productos/7</code>	Reemplazar un recurso
Modificar	PATCH	<code>/api/productos/7</code>	Cambiar parte de un recurso
Eliminar	DELETE	<code>/api/productos/7</code>	Borrar un recurso

La respuesta incluye un *código de estado* de tres dígitos que indica el resultado [5]: los de la familia 2xx señalan éxito (200 OK, 201 Created, 204 No Content); los 4xx indican un error del cliente (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found); y los 5xx un error del servidor (500 Internal Server Error). Su código deberá reaccionar de forma distinta según esta familia.

3.3. JSON: el formato de intercambio

Los datos viajan habitualmente en *JSON* (*JavaScript Object Notation*), un formato de texto ligero para representar objetos y listas, normalizado en el RFC 8259 [6]. Angular convierte

automáticamente el JSON de la respuesta en objetos de TypeScript, por lo que usted trabajará con objetos tipados y no con cadenas de texto.

3.4. Origen y CORS

Aquí aparece el concepto que más problemas causa en la práctica, así que conviene entenderlo bien.

Origen (*origin*) y política del mismo origen

El *origen* de una página es la combinación de **esquema** + **dominio** + **puerto** (por ejemplo, `http://localhost:4200`). Por seguridad, los navegadores aplican la *política del mismo origen* (*same-origin policy*): el JavaScript de una página solo puede leer libremente respuestas que provengan de su mismo origen. Si su aplicación corre en `localhost:4200` y la API en `localhost:8080`, son **orígenes distintos** (difieren en el puerto).

CORS (Cross-Origin Resource Sharing)

CORS es el mecanismo, basado en cabeceras HTTP, que permite a un servidor **autorizar** explícitamente que páginas de otros orígenes lean sus respuestas [7, 8]. El servidor concede el permiso enviando la cabecera `Access-Control-Allow-Origin`. Si esa cabecera falta, el navegador **bloquea** la lectura de la respuesta y muestra el conocido error de CORS en la consola, aunque el servidor haya respondido correctamente.

Para peticiones que pueden tener efectos secundarios (como PUT o DELETE, o POST con ciertas cabeceras), el navegador envía primero una petición *preflight* con el método OPTIONS para preguntar al servidor si la operación está permitida; solo si el servidor responde afirmativamente se envía la petición real [7]. Es fundamental comprender que **CORS se resuelve en el servidor**: el cliente no puede ínsaltárselo. En desarrollo, sin embargo, se usa un *proxy* para evitarlo, como se explica en el paso 7.

3.5. Angular moderno: las piezas que intervienen

Angular 22 favorece un modelo *standalone* (sin `NgModule`), reactividad con *señales* y ejecución *zoneless* [1]. Los términos que verá en el código son:

`HttpClient`

El servicio de Angular para realizar peticiones HTTP. Desde la v22 usa la API *Fetch* del navegador de forma predeterminada [3].

`provideHttpClient()`

La función que registra `HttpClient` en la configuración de la aplicación para poder inyectarlo.

`inject()`

La función que obtiene una dependencia (por ejemplo, `HttpClient`) sin declararla en el constructor.

`Observable`

Un flujo de datos asíncrono de la biblioteca RxJS; `HttpClient` devuelve observables a los que uno se ínsuscribe.

`signal`

Un contenedor reactivo de un valor que notifica a la vista cuando cambia; es la base de la reactividad en Angular 2026.

httpResource()

Una API estable en v22 que realiza una petición GET y expone su resultado como señales de estado (`value`, `isLoading`, `error`); reacciona automáticamente cuando cambian sus dependencias [9].

Regla práctica de la propia documentación: use `httpResource` para **lecturas** (GET) que deben reflejarse en la interfaz, y use `HttpClient` directamente para **mutaciones** (POST, PUT, PATCH, DELETE) [9].

4. Desarrollo paso a paso

Paso 1. Registrar el cliente HTTP

En una aplicación *standalone*, la configuración global vive en `app.config.ts`. Allí se registra el cliente HTTP con `provideHttpClient()`.

```
1 import { ApplicationConfig } from '@angular/core';
2 import { provideHttpClient } from '@angular/common/http';
3
4 // La configuración de la aplicación es un objeto de proveedores (providers).
5 export const appConfig: ApplicationConfig = {
6   providers: [
7     // Registra HttpClient para poder inyectarlo en servicios y componentes.
8     // En Angular 22 no hace falta withFetch(): Fetch ya es el backend por
9     // defecto.
10    provideHttpClient()
11  ]
12};
```

Listing 2: `src/app/app.config.ts` — registro del cliente HTTP.

Paso 2. Definir el modelo de datos

Se declara una *interfaz* de TypeScript que describe la forma del recurso. Tipar la respuesta evita errores y habilita el autocompletado.

```
1 // Una interfaz describe la "forma" del objeto JSON que devuelve la API.
2 // No genera código en tiempo de ejecución: solo sirve para el compilador.
3 export interface Producto {
4   id: number;
5   nombre: string;
6   precio: number;
7   disponible: boolean;
8 }
```

Listing 3: `src/app/producto.model.ts` — contrato de datos.

Paso 3. Crear el servicio REST (las cuatro operaciones)

La lógica de acceso a la API se encapsula en un *servicio*, no en el componente. Así el componente se mantiene simple y la lógica es reutilizable y testeable.

```
1 import { Injectable, inject } from '@angular/core';
2 import { HttpClient } from '@angular/common/http';
3 import { Observable } from 'rxjs';
4 import { Producto } from '../producto.model';
5
```

```

6 // @Injectable con providedIn: 'root' crea UNA sola instancia para toda la app
7
8 @Injectable({ providedIn: 'root' })
9 export class ProductoService {
10
11     // URL base del recurso. Es relativa para que funcione con el proxy (paso
12     // 7).
13     private readonly base = '/api/productos';
14
15     // inject() obtiene HttpClient sin necesitar un constructor.
16     private readonly http = inject(HttpClient);
17
18     // GET colección: devuelve un Observable con un arreglo de productos.
19     // El <Producto[]> le dice a Angular el tipo esperado de la respuesta JSON.
20     listar(): Observable<Producto[]> {
21         return this.http.get<Producto[]>(this.base);
22     }
23
24     // GET por id: obtiene un único recurso.
25     obtener(id: number): Observable<Producto> {
26         return this.http.get<Producto>(`${this.base}/${id}`);
27     }
28
29     // POST: crea un recurso. El segundo argumento es el cuerpo (body) enviado.
30     // Angular serializa el objeto a JSON automáticamente.
31     crear(p: Omit<Producto, 'id'>): Observable<Producto> {
32         return this.http.post<Producto>(this.base, p);
33     }
34
35     // PUT: reemplaza por completo el recurso identificado por id.
36     actualizar(id: number, p: Producto): Observable<Producto> {
37         return this.http.put<Producto>(`${this.base}/${id}`, p);
38     }
39
40     // DELETE: elimina el recurso. Suele responder 204 No Content (sin cuerpo).
41     eliminar(id: number): Observable<void> {
42         return this.http.delete<void>(`${this.base}/${id}`);
43     }
44 }

```

Listing 4: src/app/producto.service.ts — servicio con GET/POST/PUT/DELETE.

Omit<Producto, 'id'> es un tipo de TypeScript que representa un producto **sin** el campo id, útil al crear, porque el identificador lo asigna el servidor.

Paso 4. Mostrar datos con httpResource y señales (lectura)

Para leer y mostrar la colección se usa httpResource, que lanza la petición y expone su estado como señales: así la plantilla reacciona sola a la carga, al éxito y al error, sin suscripciones manuales.

```

1 import { Component, signal } from '@angular/core';
2 import { httpResource } from '@angular/common/http';
3 import { Producto } from '../producto.model';
4
5 @Component({
6     selector: 'app-lista-productos',
7     // Plantilla con la nueva sintaxis de control de flujo (@if / @for).
8     template: `
9         @if (productos.isLoading()) {
10             <p>Cargando...</p>
11         }
12     `
13 })

```

```

11     } @else if (productos.error()) {
12       <p>Error al cargar los productos.</p>          <!-- estado de error -->
13     } @else {
14       <ul>
15         @for (p of productos.value(); track p.id) {
16           <li>{{ p.nombre }} {{ p.precio | currency }}</li>
17         }
18       </ul>
19     }
20   '
21 })
22 export class ListaProductosComponent {
23   // httpResource realiza el GET y devuelve señales de estado.
24   // La función se re-evalúa si alguna señal interna cambia (reactividad).
25   productos = httpResource<Producto[]>(() => '/api/productos');
26 }

```

Listing 5: src/app/lista-productos.component.ts — lectura reactiva con señales.

isLoading(), error() y value() son **señales**: se invocan como funciones (con paréntesis) y la vista se actualiza automáticamente cuando su valor cambia. El track p.id del @for identifica cada elemento para que Angular redibuje solo lo necesario.

Paso 5. Ejecutar una mutación (creación) con HttpClient

Las operaciones que cambian datos se hacen con el servicio del paso 3. Como HttpClient devuelve un Observable, hay que *suscribirse* para que la petición se ejecute realmente.

```

1 import { Component, inject } from '@angular/core';
2 import { ProductoService } from '../producto.service';
3
4 @Component({ /* ... */ })
5 export class NuevoProductoComponent {
6   private readonly servicio = inject(ProductoService);
7
8   guardar(): void {
9     const nuevo = { nombre: 'Teclado', precio: 25.9, disponible: true };
10
11     // subscribe() DISPARA la petición. Sin él, el Observable no se ejecuta.
12     this.servicio.crear(nuevo).subscribe({
13       next: (creado) => console.log('Creado con id', creado.id), // 201
14         Created
15       error: (e)      => console.error('No se pudo crear:', e.status)
16     });
17   }
18 }

```

Listing 6: Fragmento de componente — crear un producto.

Paso 6. Manejo de errores

Conviene centralizar el tratamiento de errores. El objeto de error de Angular (HttpResponseError) trae el código de estado en status, lo que permite reaccionar según la familia 4xx o 5xx [5].

```

1 import { catchError, throwError } from 'rxjs';
2 import { HttpResponseError } from '@angular/common/http';
3
4 listarSeguro() {
5   return this.http.get<Producto[]>(this.base).pipe(
6     // pipe() encadena operadores; catchError intercepta el fallo.

```

```

7      catchError((err: HttpResponse) => {
8          if (err.status === 0)      console.error('Sin red o bloqueo CORS');
9          else if (err.status === 404) console.error('Recurso no encontrado');
10         else if (err.status >= 500) console.error('Error del servidor');
11         // Re-emite el error para que quien se suscriba pueda reaccionar.
12         return throwError(() => err);
13     })
14 );
15 }

```

Listing 7: Manejo de errores con el operador `catchError` de RxJS.

Un `status` igual a 0 es la señal típica de un **bloqueo por CORS** o de ausencia de red: el navegador impidió leer la respuesta antes de que llegara un código real. Es el error que resolverá en el paso siguiente.

Paso 7. Resolver CORS en desarrollo con un *proxy*

Durante el desarrollo, la aplicación (`localhost:4200`) y la API (por ejemplo, `localhost:8080`) tienen orígenes distintos, por lo que el navegador bloquea la lectura si el servidor no envía las cabeceras CORS [7]. La solución limpia en desarrollo es un *proxy*: la CLI de Angular reenvía a la API todas las peticiones que empiezan por `/api`, de modo que, para el navegador, todo procede del mismo origen `localhost:4200` y no hay CORS que resolver.

```

1 {
2   "/api": {
3     "target": "http://localhost:8080", // dirección real de la API
4     "secure": false,                  // permite destinos sin HTTPS válido
5     "changeOrigin": true              // ajusta la cabecera Host al
      destino
6   }
7 }

```

Listing 8: `proxy.conf.json` — redirige `/api` hacia la API real.

```

1 ng serve --proxy-config proxy.conf.json

```

Listing 9: Arrancar el servidor con el proxy.

CORS en producción

El *proxy* es una comodidad de desarrollo, **no** una solución de producción. En producción, el servidor de la API debe habilitar CORS de forma explícita (enviando `Access-Control-Allow-Origin` con el origen autorizado) o bien servir el *frontend* y la API bajo el mismo origen [7, 8]. Nunca desactive la seguridad del navegador para resolver un CORS.

5. Buenas prácticas

Mantenga toda la lógica HTTP dentro de servicios y nunca en los componentes; use rutas relativas (`/api/...`) para que el proxy funcione y para no fijar el dominio en el código; tipe siempre las respuestas con interfaces; prefiera `httpResource` para lecturas que alimentan la vista y `HttpClient` para mutaciones; y trate los errores por familia de código de estado en lugar de suponer que la petición siempre tiene éxito.

6. Actividad entregable

Construya una pequeña aplicación Angular 22 que consuma una API REST pública o propia y que permita: (a) listar una colección con `httpResource`, mostrando los estados de carga y error; (b) crear un recurso con POST; (c) eliminar un recurso con DELETE y refrescar la lista; y (d) funcionar con un `proxy.conf.json` correctamente configurado. Entregue el repositorio con un README que indique las versiones de Node y Angular usadas (evidenciando `ng version`) y una captura del error de CORS antes de aplicar el proxy junto con la vista ya funcionando después de aplicarlo.

Criterio de evaluación	Puntos
Configuración correcta del cliente HTTP y del proxy	2.0
Servicio con las operaciones GET/POST/DELETE tipadas	3.0
Lectura reactiva con <code>httpResource</code> (estados carga/error)	2.5
Manejo de errores por código de estado	1.5
Evidencia de diagnóstico y resolución del CORS	1.0
Total	10.0

Referencias

- [1] Angular Team. *Announcing Angular v22*. <https://blog.angular.dev/announcing-angular-v22-c52bb83a4664>. Angular Blog. Publicado el 3 de junio de 2026. 2026.
- [2] Angular Team. *Version compatibility*. <https://angular.dev/reference/versions>. Google. Consultado en julio de 2026. 2026.
- [3] Angular Team. *Making HTTP requests*. <https://angular.dev/guide/http>. Google. Consultado en julio de 2026. 2026.
- [4] Roy Thomas Fielding. “Architectural Styles and the Design of Network-based Software Architectures”. Ph.D. dissertation. Irvine, CA, USA: University of California, Irvine, 2000.
- [5] Roy T. Fielding, Mark Nottingham y Julian Reschke. *HTTP Semantics*. Request for Comments 9110. Internet Engineering Task Force (IETF), jun. de 2022. DOI: [10.17487/RFC9110](https://doi.org/10.17487/RFC9110).
- [6] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. Request for Comments 8259. Internet Engineering Task Force (IETF), dic. de 2017. DOI: [10.17487/RFC8259](https://doi.org/10.17487/RFC8259).
- [7] MDN Web Docs. *Cross-Origin Resource Sharing (CORS)*. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>. Mozilla. Consultado en julio de 2026. 2026.
- [8] WHATWG. *Fetch Standard*. <https://fetch.spec.whatwg.org/>. Living Standard. Consultado en julio de 2026. 2026.
- [9] Angular Team. *httpResource: reactive data fetching with signals*. <https://angular.dev/guide/http/http-resource>. Google. Consultado en julio de 2026. 2026.